Phase 1: React Fundamentals + Static Watchlist

What we built

- A **Watchlist** of 8 coins (mock data) with a **search box**
- Clicking a coin **selects** it. The header, the middle panel and the browser tab title all update.
- The middle panel shows the selected coin's price and 24h stats (the chart replaces this in Phase 4)

Files and what each one does

File	Role
`src/types.ts`	TypeScript shape of a `Ticker` (symbol, price, change…)
`src/data/mockTickers.ts`	Fake hard-coded data (replaced by live Binance data in Phase 2–3)
`src/utils/format.ts`	Pure helper functions: `67250.5 → "67,250.50"`, `2.34 → "+2.34%"`
`src/components/Panel.tsx`	Reusable card. Shows **composition** with `children`
`src/components/Header.tsx`	Top bar. Only **reads** props
`src/components/Watchlist.tsx`	List + search. Shows **local state**, **controlled input**, **lists & keys**
`src/components/WatchlistRow.tsx`	One row. Gets data via props and reports clicks via a callback
`src/components/SymbolSummary.tsx`	Selected coin details (placeholder for the chart)
`src/App.tsx`	Root. **Owns the shared state** (`selectedSymbol`) and lays out the page

Component tree

```
App                     ← owns: selectedSymbol (state)
├── Header              ← reads selectedSymbol
└── main
    ├── Panel "Watchlist"
    │   └── Watchlist    ← owns: search (state); calls onSelect
    │       └── WatchlistRow × N   (key = symbol)
    ├── Panel "Chart"
    │   └── SymbolSummary ← reads selected ticker
    └── Panel "Order Panel"
```

Concepts (theory only)

1. Component

A **function that returns UI**. You build a big UI out of small, reusable pieces. Names start with a capital letter (`Watchlist`), so React can tell them apart from HTML tags (`div`).

2. JSX

HTML-like syntax inside JS/TS: `<h1>{title}</h1>` . - `{ }` drops a JS expression into the markup. - `className` instead of `class` (because `class` is a reserved word in JS). - It compiles into plain function calls (`React.createElement`). The browser never sees JSX.

3. Props

Inputs to a component, passed from parent to child like function arguments. - **Read-only:** a child must never modify its props. - Can be any value: strings, objects, and even **functions** (`onSelect`). Passing a function down is how a child "talks back" to its parent. - `children` is a special prop: whatever you put between the opening and closing tags.

4. State (`useState`)

A component's **memory**: data that changes over time and should update the screen.

```
const [search, setSearch] = useState('')
```

- `search` is the current value. `setSearch` is the function that changes it.
- Calling the setter **schedules a re-render** with the new value.
- Never change state directly (`search = 'x'` won't update the screen). Always use the setter.

Props vs state

	Props	State
Who owns it?	The parent	The component itself
Can it change?	Only if the parent passes a new value	Yes, via the setter
Analogy	Function arguments	Local variables that survive between renders

5. Re-rendering: what triggers it

A component re-renders when: 1. Its **state** changes 2. Its **parent re-renders** (so it gets new props) 3. A **context** it uses changes (later)

"Re-render" = React calls your function again, compares the new UI with the old one (**reconciliation** against the **Virtual DOM**), and changes **only what differs** in the real DOM. Updating the real DOM is the slow part, which is why React keeps it to a minimum.

6. Rendering lists & `key`

`tickers.map(t => <WatchlistRow key={t.symbol} ... />)` turns an array of data into an array of elements. - `key` gives each item a **stable, unique identity**, so when the list is filtered or reordered, React knows *which* row moved, appeared or disappeared, and can reuse it instead of rebuilding it. - Don't use the array index as the key when the list can change order or be filtered. It causes wrong rows to keep the wrong state.

7. Lifting state up

Two siblings need the same data (Watchlist **sets** the selection, Header and SymbolSummary **show** it). → Move the state to their **closest common parent** (App). Pass the value down as props, and pass the setter down as a callback. This is **one-way data flow**: data goes down, events go up. *(When too many layers have to pass props along, that's "prop drilling". Phase 3 fixes it with a global store, Zustand.)*

8. Controlled input

The input's value **comes from state**, and every keystroke **updates state**:

```
<input value={search} onChange={e => setSearch(e.target.value)} />
```

React is the "single source of truth". Validation and filtering become easy (we'll use this for the order form in Phase 5).

9. Derived data

`filtered` and `selectedTicker` are **calculated during render** from props and state, not stored in their own state. Rule: *if you can compute it, don't store it.* This avoids two copies of the same data getting out of sync.

10. `useEffect` (basic idea)

Lets a component **sync with something outside React**: the tab title, a timer, a WebSocket (Phase 2).

```
useEffect(() => { document.title = ... }, [selectedSymbol])
```

- It runs **after** the render.
- The **dependency array** `[selectedSymbol]` means "re-run only when this value changes".
- It can return a **cleanup function** (close the socket, clear the timer). This is very important in Phase 2.

11. Conditional rendering

`{filtered.length === 0 && <p>No symbols match</p>}` shows the element only if the condition is true. Another form is the ternary: `isUp ? 'text-up' : 'text-down'`.

12. Presentational vs container components

- **Presentational** (WatchlistRow, Header): only show props and have no state. Easy to reuse and test.
- **Container / stateful** (App, Watchlist): own the state and decide what data the children get.

Try it yourself

1. Type `eth` in the search box: the list filters as you type (state → re-render → derived `filtered`).
2. Click SOL: the header, the middle panel and the **browser tab title** change (the effect at work).
3. In `mockTickers.ts`, change BTC's `changePercent` to `-3`, save, and watch it turn red (HMR).

Resume mapping

- "**React**", "**TypeScript**": components, props, state and typed props.
- "Built a trading web app with a **live watchlist**": the watchlist UI is done (it goes "live" in Phase 3).

Interview Qs

1. **Props vs state?** Props are read-only inputs from the parent. State is the component's own changeable data, and updating it triggers a re-render.
2. **Why do lists need keys?** So React can match items between renders and update only what changed. Use stable ids, not indexes, when the list can reorder.
3. **What causes a re-render?** A state change, a parent re-render, or a change in a context it uses.
4. **What is the Virtual DOM?** A lightweight JS copy of the UI. React compares (diffs) the old and new copies and applies only the minimum changes to the real DOM.
5. **How do sibling components share data?** Lift the state to their common parent. Or, when it's needed all over the app, use a global store (Zustand/Redux) or Context.
6. **What is a controlled component?** A form input whose value is driven by React state, so React is the single source of truth.
7. **What is `useEffect` for?** Side effects: syncing with things outside React (DOM APIs, timers, network, sockets). The dependency array controls when it re-runs, and the cleanup function undoes it.